

INF221 — Lecture 12

Today's topics

1. Working with graphs
 - A. Recap: Graph representation
 - B. Breadth-first search
 - C. Depth-first search
 - D. Topological sort
 2. Limits of computability
 - A. Das Entscheidungsproblem
 - B. $P = NP?$
-

Administrativa

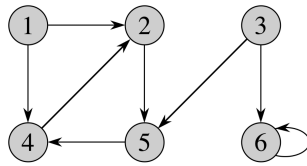
- Exercise session Thursday 7 May, 10.15–12.00 in TF1-105
 - Quick review of material
 - Exercises
 - Q & A Thursday 21 May, 10.00–11.00, in Plesser's office (TF2-305)
 - Exam Wednesday 27 May, 09.00–12.30
 - Open book exam
 - You can take along Thomas Cormen, *Algorithms Unlocked*, with any *handwritten* notes *on the pages of the book*.
 - No loose sheets or glued-in print-outs allowed!
 - The sheet with mathematical expressions provided with earlier exams will also be included in this year's exam.
 - You will be given a calculator.
-

Recap: Graph representation

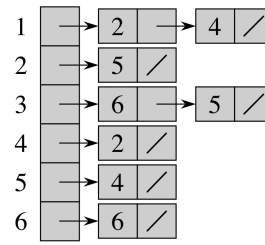
See Compendium, part 18

- Let $G = (V, E)$ be a graph
 - V is the set of vertices
 - E is the set of edges

Directed graph



(a)



(b)

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

(c)

CLRS, Ch 22

A quick analysis of storage requirements

- The figures show a graph with $V = 6$ nodes and $E = 8$ edges
- In the adjacency-list representation, we need
 - An array with $V = 6$ spaces for list heads (pointers), usually 8 Bytes each, thus 48B in total
 - One list entry for each of $E = 8$ edges, each consisting of the node ID and a pointer to the next list element; each of these typically takes 8B, so in total we have $8 \times 2 \times 8B = 128B$
 - Altogether, we need $48B + 128B = 172B$
- In the adjacency matrix, we have $V^2 = 6^2 = 36$ entries.
 - If we represent each entry with one byte, we'd need $36B$
 - But we only need a single bit for each edge, so $36\text{bit} = 4.5B$ would suffice

A dense graph

- Assume that each graph has 5 outgoing edges.
- Then we have 30 edges in total, each needing $16B$, so $480B$ for the edges
- Together with the array for the list heads, we need $528B$ for the adjacency-list representation
- The size of the adjacency matrix does not change, we can still pack it into $4.5B$

A large, sparse graph

- Consider the network of all roads and paths in Norway (you [can download it here](#))
- It contains several million intersections (vertices), but only a very small number of edges per intersection (I haven't checked yet, but likely no more than five, I'd think).
- Lets assume we have 1 million vertices with 5 edges each, so 5 million edges.
- Then for the adjacency lists we have

$$10^6 \times 8B + 10^6 \times 5 \times 16B = 88 \times 10^6 B \approx 100MB$$

- For the adjacency matrix, on the other hand, we have

$$10^6 \times 10^6 \times 1\text{bit} = 10^{12}\text{bit} \approx 10^{11}B = 100 \times 10^9 B = 100GB$$

In this case, the adjacency matrix takes 1000 times more space than the list representation.

More reading

For more on efficient storage and representation of complex graphs, see the papers by [Kunkel et al \(2014\)](#) and [Jordan et al \(2018\)](#).

In-class exercise

1. Write down adjacency lists and adjacency matrix for graph on the whiteboard

Graph	Adjacency List	Adjacency Matrix																																																																																																														
<pre>graph TD; s((s)) --> a((a)); s((s)) --> c((c)); a((a)) --> e((e)); b((b)) --> a((a)); b((b)) --> e((e)); c((c)) --> e((e)); c((c)) --> g((g)); e((e)) --> b((b)); e((e)) --> h((h)); f((f)) --> c((c)); f((f)) --> g((g)); g((g)) --> f((f)); g((g)) --> h((h)); g((g)) --> i((i)); h((h)) --> g((g)); i((i)) --> h((h));</pre>	<u>Adjacency list</u> <table><tr><td>$s \rightarrow a, c$</td><td>$f \rightarrow c$</td></tr><tr><td>$a \rightarrow e$</td><td>$g \rightarrow f, h, i$</td></tr><tr><td>$b \rightarrow a$</td><td>$h \rightarrow g$</td></tr><tr><td>$c \rightarrow e, g$</td><td>$i \rightarrow h$</td></tr><tr><td>$e \rightarrow b, h$</td><td></td></tr></table>	$s \rightarrow a, c$	$f \rightarrow c$	$a \rightarrow e$	$g \rightarrow f, h, i$	$b \rightarrow a$	$h \rightarrow g$	$c \rightarrow e, g$	$i \rightarrow h$	$e \rightarrow b, h$		<u>Adjacency matrix</u> <table><tr><th></th><th>s</th><th>a</th><th>b</th><th>c</th><th>e</th><th>f</th><th>g</th><th>h</th><th>i</th></tr><tr><th>s</th><td></td><td>1</td><td></td><td>1</td><td></td><td></td><td></td><td></td><td></td></tr><tr><th>a</th><td></td><td></td><td></td><td></td><td>1</td><td></td><td></td><td></td><td></td></tr><tr><th>b</th><td></td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><th>c</th><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td>1</td><td></td></tr><tr><th>e</th><td></td><td></td><td></td><td>1</td><td></td><td></td><td></td><td></td><td>1</td></tr><tr><th>f</th><td></td><td></td><td></td><td>1</td><td></td><td></td><td></td><td></td><td></td></tr><tr><th>g</th><td></td><td></td><td></td><td></td><td></td><td></td><td>1</td><td>1</td><td>1</td></tr><tr><th>h</th><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td>1</td><td></td></tr><tr><th>i</th><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td>1</td></tr></table>		s	a	b	c	e	f	g	h	i	s		1		1						a					1					b		1								c								1		e				1					1	f				1						g							1	1	1	h								1		i									1
$s \rightarrow a, c$	$f \rightarrow c$																																																																																																															
$a \rightarrow e$	$g \rightarrow f, h, i$																																																																																																															
$b \rightarrow a$	$h \rightarrow g$																																																																																																															
$c \rightarrow e, g$	$i \rightarrow h$																																																																																																															
$e \rightarrow b, h$																																																																																																																
	s	a	b	c	e	f	g	h	i																																																																																																							
s		1		1																																																																																																												
a					1																																																																																																											
b		1																																																																																																														
c								1																																																																																																								
e				1					1																																																																																																							
f				1																																																																																																												
g							1	1	1																																																																																																							
h								1																																																																																																								
i									1																																																																																																							

2. Draw directed graph from adjacency list
3. Draw directed graph from adjacency matrix

Breadth-first search

See Compendium Part 18.

Algorithm

Input

- Graph $G = (V, E)$, directed or undirected
- One source vertex $s \in V$

Output

- For all $v \in V$, the distance $v.d$ from s to v , i.e., the smallest number of edges we need to traverse to get from s to v

- For each v also $v.\pi$, the predecessor of v on a shortest path from s to v , i.e., the vertex u such that (u, v) is the last edge on a shortest path from s to v
- The set of all edges

$$\{(v.\pi, v) | v \neq s\}$$

forms a tree

Idea

- Let paint flow (slowly) from s through the graph
- First reaches all vertices at distance 1 from s
- Then reaches all vertices at distance 2 from s and so on
- Need to keep an overview over which vertices are currently at the front of the paint wave
 - Each vertex will be at the front exactly once
- Use a FIFO queue Q to keep overview over "wavefront"
 - $v \in Q$ if and only if paint has reached v but no paint has yet flowed out of v

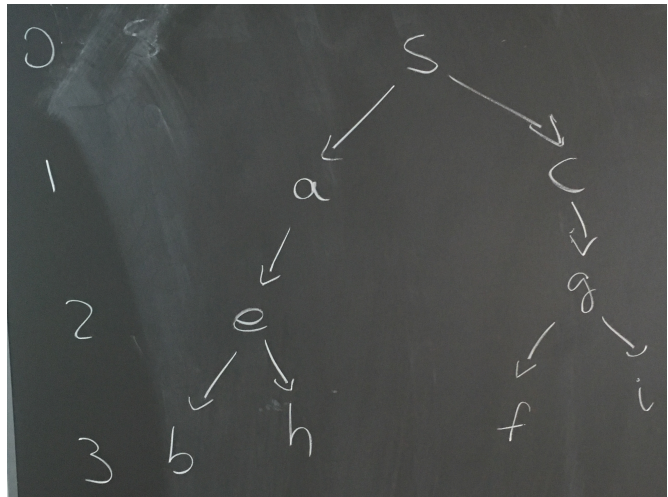
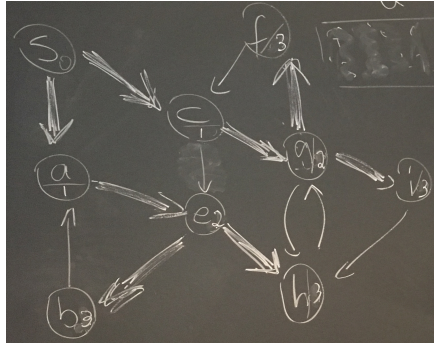
Implementation

```

BFS(V, E, s)
  for u in V
    u.d = inf
    u.p = NIL
  s.d = 0
  Q = new Queue
  Enqueue(Q, s)
  while Q not empty
    u = Dequeue(Q)
    for v in G.Adj[u]
      if v.d == inf      # v has not been "painted" yet
        v.d = u.d + 1
        v.p = u
        Enqueue(Q, v)

```

Example



Analysis

Run-time

- Set-up is $O(V)$
- Each vertex is enqueued at most once
 - Enqueuing is $O(V)$
- Each vertex is dequeued at most once
 - Dequeuing is $O(V)$
- Main loop
 - Each vertex is dequeued once as u
 - For given u , we run over all adjacent v , i.e., all edges (u, v) starting at u
 - In total, we run over each edge once
 - The nested loop is just executed $O(E)$ times
- Total runtime $O(V + E)$

Correctness

- Obviously correct if we just have s
- First round through while-loop
 - Enqueue all vertices adjacent to s , all get $v.d = 1$
 - At end, Q contains all vertices adjacent to s with $v.d = 1$
- Subsequent rounds through while loop
 - We dequeue vertices with $u.d = 1$ from the head of the queue
 - We enqueue their adjacent vertices with $v.d = 2$ at tail of queue (unless they have $v.d=1$ from before)
 - Queue will have vertices with $d = 1$ at beginning, $d = 2$ towards end
 - We need to dequeue all vertices with $d = 1$ before enqueuing the first with $d = 3$
 - Therefore, Q will at most contain two different values of d , with the lowest one always first

- Each vertex gets assigned d only once, thus vertices visited later are assigned increasing distances
 - "visited later" means "more edges needed to be traversed": d represents distance
- See Compendium for detailed proof (not part of the curriculum)

Printing a path from s to v

- We cannot start this process from s : Any outgoing edge could lead toward v —which one should be choose?
- Can we work our way back from v ?
- Assume we have performed a breadth-first search
 - Then we have constructed a tree: a *breadth-first tree* (see Compendium for proof)
 - Tree structure is represented in predecessor information $u.\pi$
- We can start at v , follow the chain of predecessors and see if that takes us to s

$$v \longrightarrow v.\pi \longrightarrow v.\pi.\pi \longrightarrow \cdots \overset{?}{\longrightarrow} s$$

- Will this chain always end after a finite number of steps (no infinite loop)?
 - Will we always get to s ?
 - What will happen if not?
- The following algorithm will print the path from s to a given node v

```
Print-Path( $G, s, v$ )
  if  $v == s$ 
    print  $s$ 
  elseif  $v.p == \text{NIL}$ 
    print "no path from  $s$  to  $v$ "
  else
    Print-Path( $G, s, v.p$ )
    print  $v$ 
```

Example

See whiteboard.

Depth-first search

Compendium, part 19

Algorithm

Input

Graph $G = (V, E)$, directed or undirected

Output

For each vertex $v \in V$

- the *discovery time* $v.d$
- the *finishing time* $v.f$
- the predecessor $v.\pi$

Idea

- Explore every edge in graph
- May require starting from different vertices
- Once we get to a vertex, explore all its adjacent vertices
 - go into depth first
 - recursive approach
- We *color* vertices while algorithm is running
 - **white**: vertex is undiscovered
 - **gray**: discovered, but not yet finished (exploration not complete)
 - **black**: finished (found all vertices reachable from this vertex)
- Discovery and finishing times
 - Unique integers from 1 to $2|V|$
 - For all v , $v.d < v.f$
 - Thus $1 \leq v.d < v.f \leq 2|V|$

Implementation

global time

DFS(G)

```
for u in G.V
    u.c = WHITE
    u.p = NIL
    time = 0

for u in G.V
    if u.c == WHITE
        DFS-VISIT(G, u)
```

DFS-VISIT(G, u)

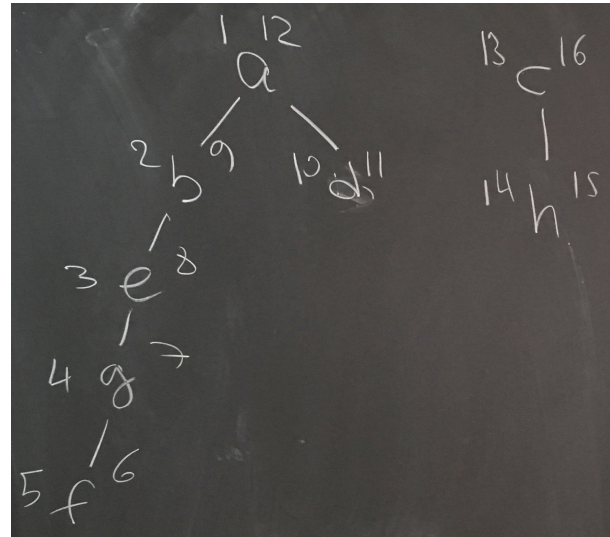
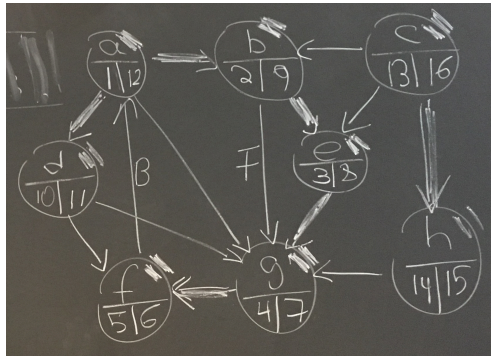
```
time = time + 1
u.d = time
u.c = GRAY
for each v in G.Adj[u]
    if v.color == WHITE
        v.p = u
        DFS-VISIT(G, v)
```

```

u.c = BLACK
time = time + 1
u.f = time

```

Example



Analysis

Run-time

- $\Theta(V + E)$
- Each vertex and each edge is examined once

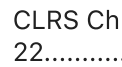
Depth-first forests

- Depth-first search creates a forest with at least one tree
- Each tree is a depth-first tree
- A depth-first tree is composed of edges (u, v) where u is gray and v is white when (u, v) is explored

Parenthesis theorem

For all vertices u and v , exactly one of the following holds after DFS is complete, considering u and v as nodes of a depth-first tree:

1. u "happens" before v ($u.d < u.f < v.d < v.f$) or v "happens" after u ($v.d < v.f < u.d < u.f$) and neither of u and v is a descendant of each other.
2. v "happens" while u is ongoing: $u.d < v.d < v.f < u.f$ and v is a descendant of u
3. u "happens" while v is ongoing: $v.d < u.d < u.f < v.f$ and u is a descendant of v



Classification of edges

We can group edges in a graph into four groups based on the result of a depth-first search:

- **Tree edge:** an edge in the depth-first forest, found when exploring (u, v)
- **Back edge:** (u, v) where u is a descendant of v , i.e., $[u.d, u.f] \subset [v.d, v.f]$
- **Forward edge:** (u, v) where v is a descendant of u but not a tree edge
- **Cross edge:** any other edge, may also connect edges in different trees

Theorem

In DFS of an *undirected* graph we can only get tree and back edges.

Visualizing Depth-First Search

The following requires package pydot installed

The code is not part of the curriculum, it is only provided here to illustrate the DFS process.

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
import pydot
import IPython.display as IPd
```

Adjacency list for example graph from DFS example above

```
In [2]: Gal = {'a': ['b', 'd'],
               'b': ['e', 'g'],
               'c': ['b', 'e', 'h'],
               'd': ['f', 'g'],
               'e': ['g'],
               'f': ['a'],
               'g': ['f'],
               'h': ['g']}
```

Implementation

```
In [3]: class DFSVertex:
        """Represent one vertex."""

        def __init__(self, label):
            self.n = label      # vertex name
            self.d = np.inf     # discovery time
            self.f = np.inf     # finishing time
            self.p = None       # parent

        def __repr__(self):
            p = self.p if self.p is not None else self.p.n
            return f'({self.n}, p: {p}, d: {self.d}, f: {self.f})'

        def is_white(self):
            return self.d == np.inf

        def is_gray(self):
            return self.d < np.inf and self.f == np.inf

        def is_black(self):
            return self.f < np.inf

        def is_root(self):
            return self.p is None and self.d < np.inf

        def render(self, plain=False, **kwargs):
            """Return pydot Node object with proper attributes."""

            if plain:
                return pydot.Node(self.n, label=f'{self.n}', fontsize=10, **kwargs)

            c = 'black'
            bc = 'white'
            fc = 'black'
```

```

fn = 'times'

vd = '&infin;';
vf = '&infin;';
if self.is_gray():
    bc = 'gray'
    vd = self.d
if self.is_black():
    bc = 'black'
    fc = 'white'
    fn = 'times bold'
    vd = self.d
    vf = self.f

if self.is_root():
    c = 'red'

return pydot.Node(self.n, label=f'{self.n}|{{{vd}}}|{vf}}',
                  shape='record', fillcolor=bc, fontcolor=fc,
                  fontname=fn, color=c, style='filled', fontsize=10)

```

```

In [4]: class DFSEdge:
        """Represent one edge."""

        def __init__(self, u, v):
            self.u = u
            self.v = v
            self.tree_edge = False

        def render(self, plain=False):
            """Return pydot Edge object with proper attributes."""
            c = 'red' if self.tree_edge and not plain else 'black'
            return pydot.Edge(self.u.n, self.v.n, color=c)

```

```

In [5]: class DFSGraph:
        """Represent entire graph."""

        def __init__(self, adj_lists, directed=True):
            self.directed = directed
            self.V = {u: DFSVertex(u) for u in adj_lists.keys()}
            self.adj = {u: [DFSEdge(self.V[u], self.V[v]) for v in adj]
                        for u, adj in adj_lists.items()}
            if not directed:
                for u, adj in adj_lists.items():
                    for v in adj:
                        self.adj[v].append(DFSEdge(self.V[v], self.V[u]))
            self.time = None
            self._silent = False

        def run_dfs(self, ordered_vertices=None, silent=False):
            """
            :param ordered_nodes: list of vertices; vertices will be visited in
            :param silent: if True, intermediate steps will not be shown
            """

```

```

    if ordered_vertices is None:
        ordered_vertices = self.V.values()
    else:
        assert set(ordered_vertices) == set(self.V.values())

    self._silent = silent

    self.render()

    self.time = 0

    for u in ordered_vertices:
        if u.is_white():
            self.dfs_visit(u)

    self._silent = False # restore default

def dfs_visit(self, u):
    self.time += 1
    u.d = self.time
    self.render()

    for e in self.adj[u.n]:
        v = e.v
        if v.is_white():
            v.p = e.u
            e.tree_edge = True
            self.dfs_visit(v)

    self.time += 1
    u.f = self.time
    self.render()

def render(self):
    """Displays graph"""

    if self._silent:
        return

    g = pydot.Dot(rankdir="LR")
    if not self.directed:
        g.set_edge_defaults(dir='none')
    for u in self.V.values():
        g.add_node(u.render())

    # build set of edges to render
    if self.directed:
        edges = (e for adj in self.adj.values() for e in adj)
    else:
        # keep only one per u-v pair; if one is tree edge, keep that
        ed = {}
        for adj in self.adj.values():
            for e in adj:
                uv = tuple(sorted((e.u.n, e.v.n)))
                if uv not in ed or (e.tree_edge and not ed[uv].tree_edge):
                    ed[uv] = e

```

```

        edges = ed.values()

        for e in edges:
            g.add_edge(e.render())

        IPd.display(IPd.Image(g.create_png()))

    def render_forest(self):
        """Display depth-first forest."""

        gtype = 'digraph' if self.directed else 'graph'
        g = pydot.Dot(rankdir='LR', graph_type=gtype)
        for u in self.V.values():
            g.add_node(u.render())

        for u in self.V.values():
            if u.p is None:
                self.render_subtree(g, u)

        IPd.display(IPd.Image(g.create_png()))

    def render_subtree(self, g, u):
        for e in self.adj[u.n]:
            if e.tree_edge:
                g.add_edge(e.render())
                self.render_subtree(g, e.v)

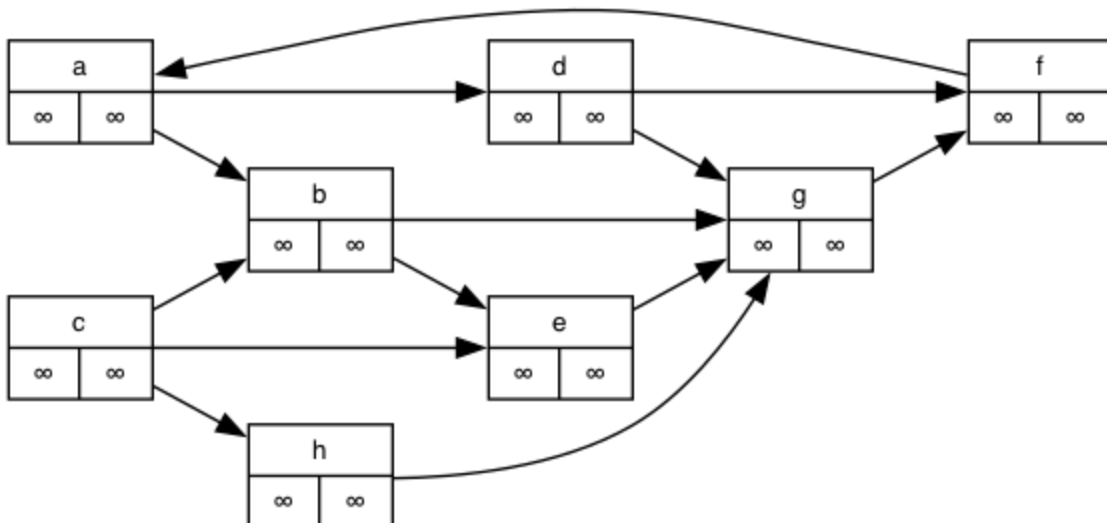
```

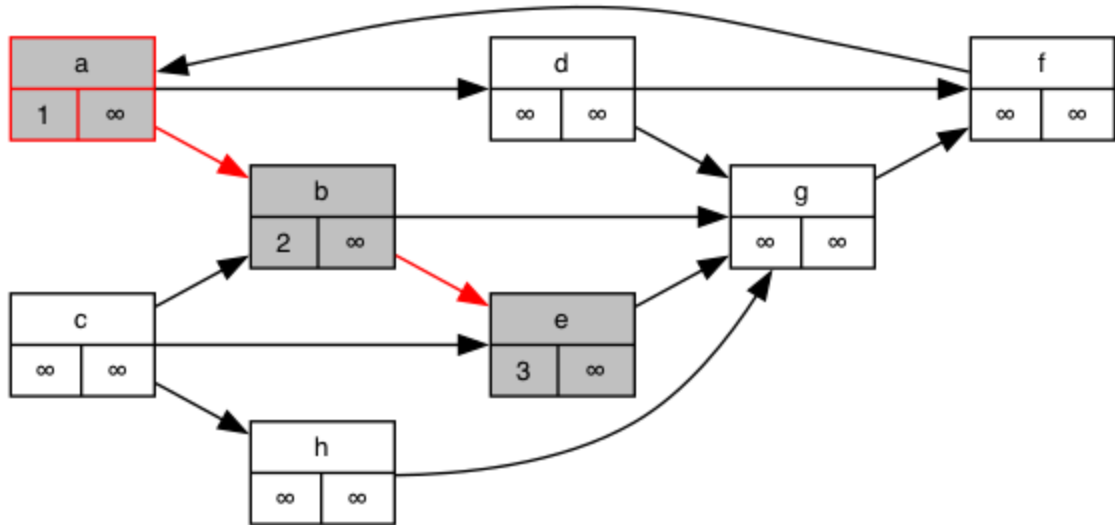
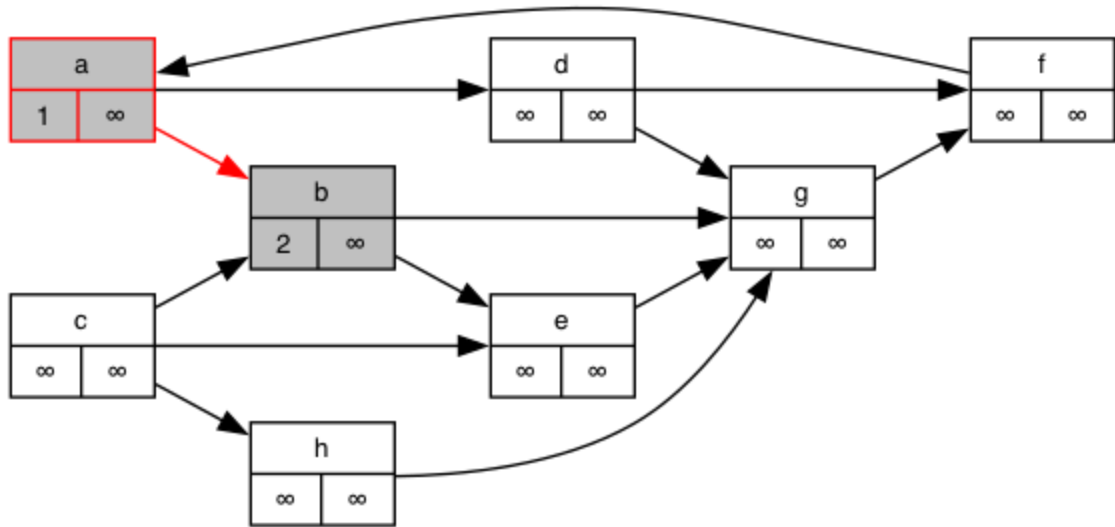
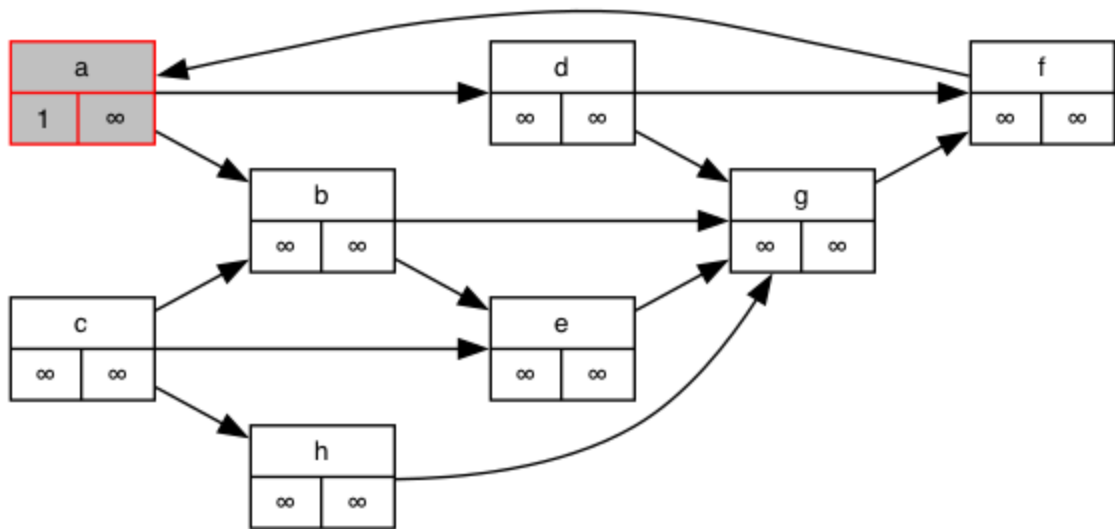
Run algorithm

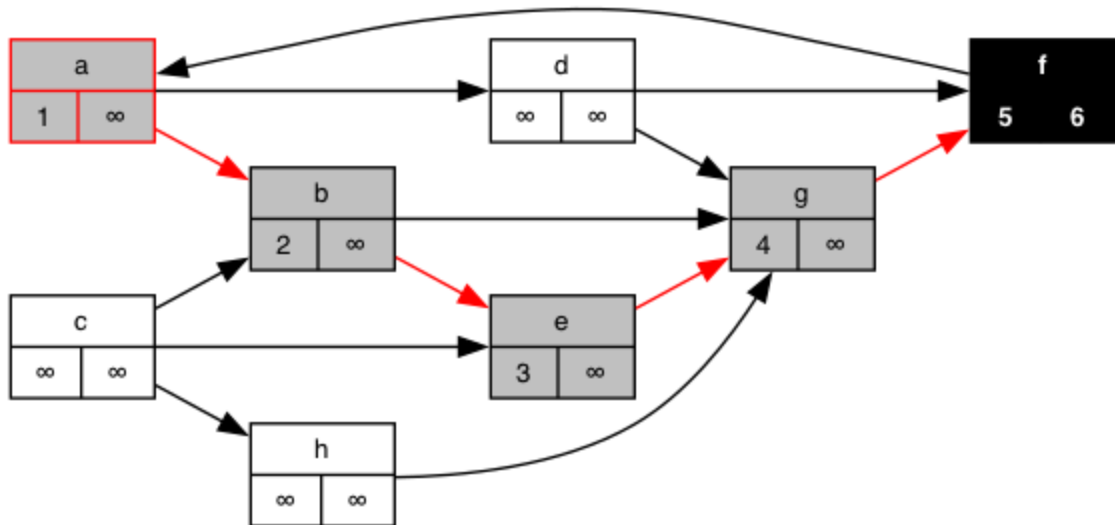
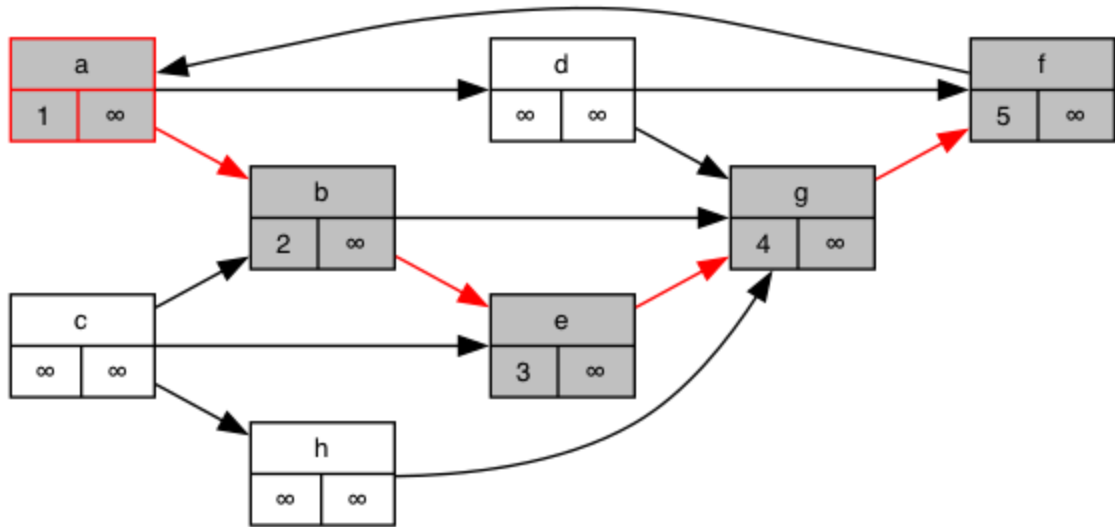
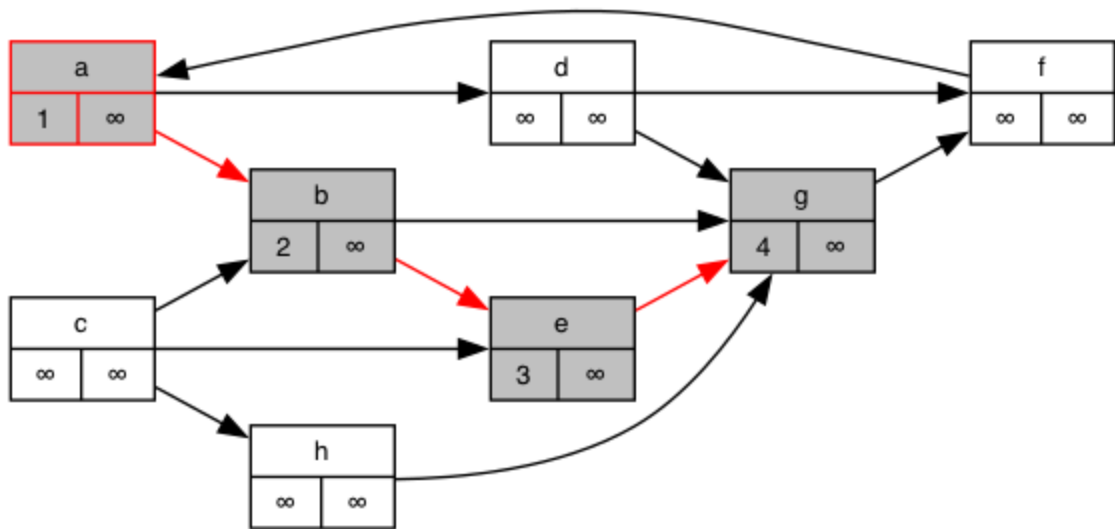
```

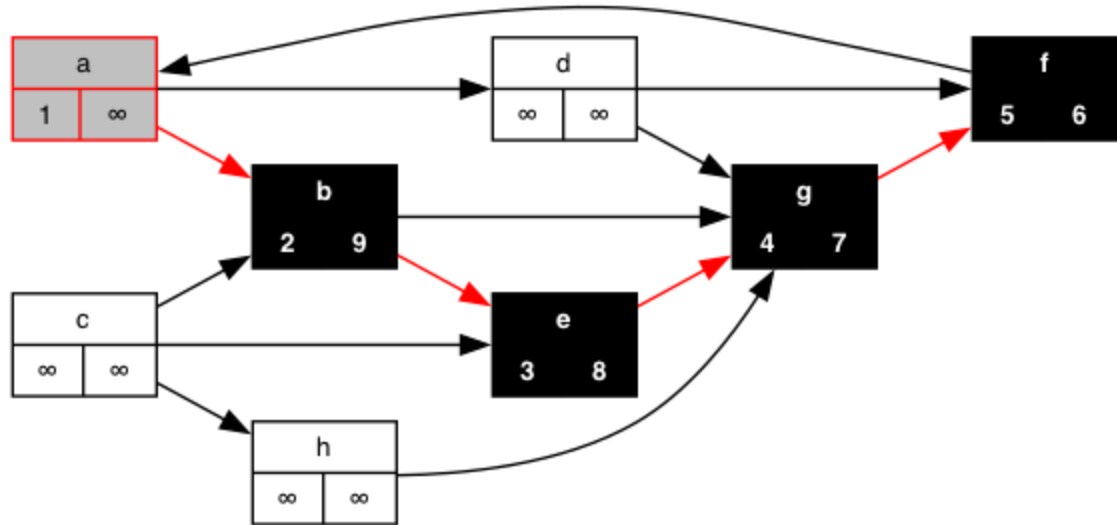
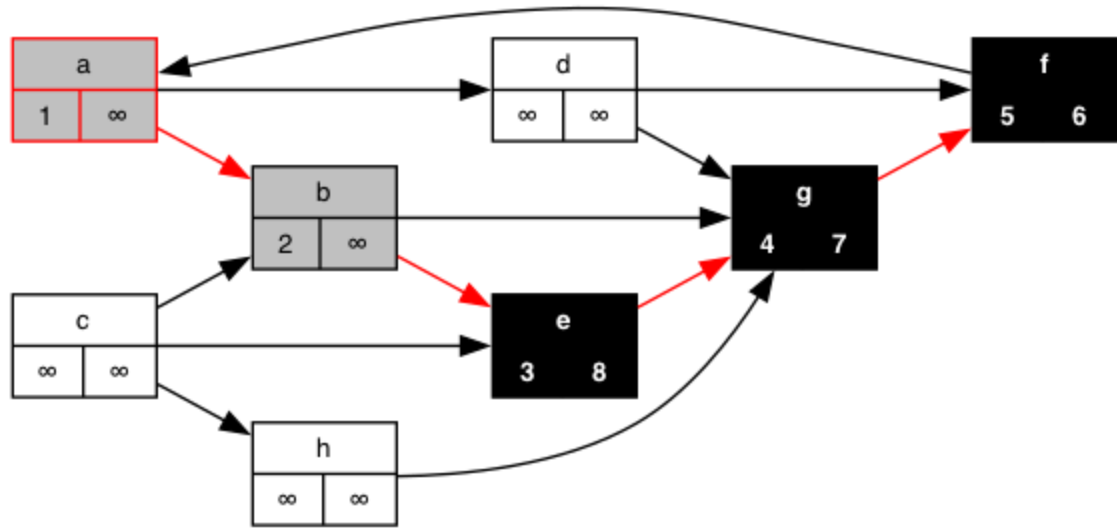
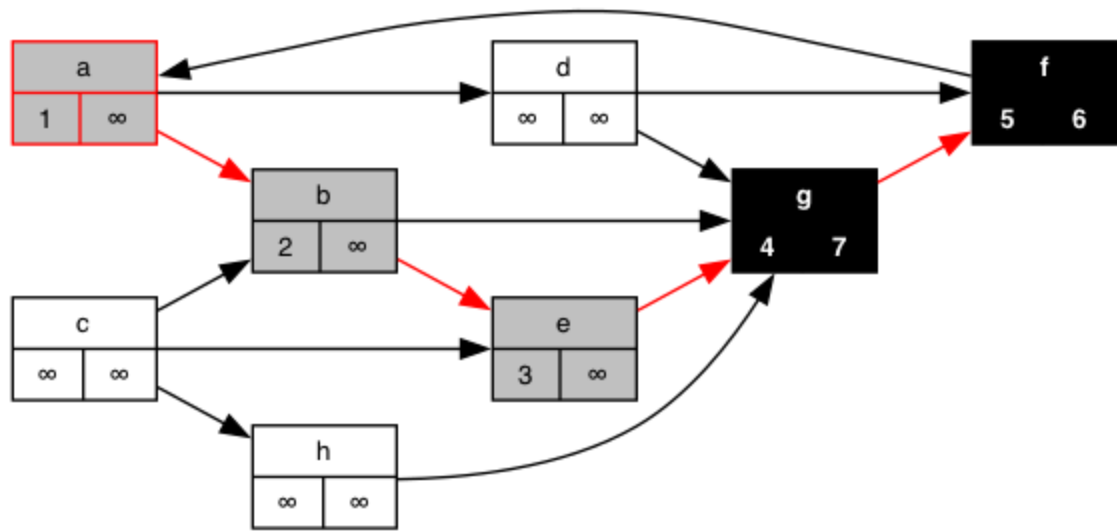
In [6]: G = DFSGraph(Gal)
        G.run_dfs()

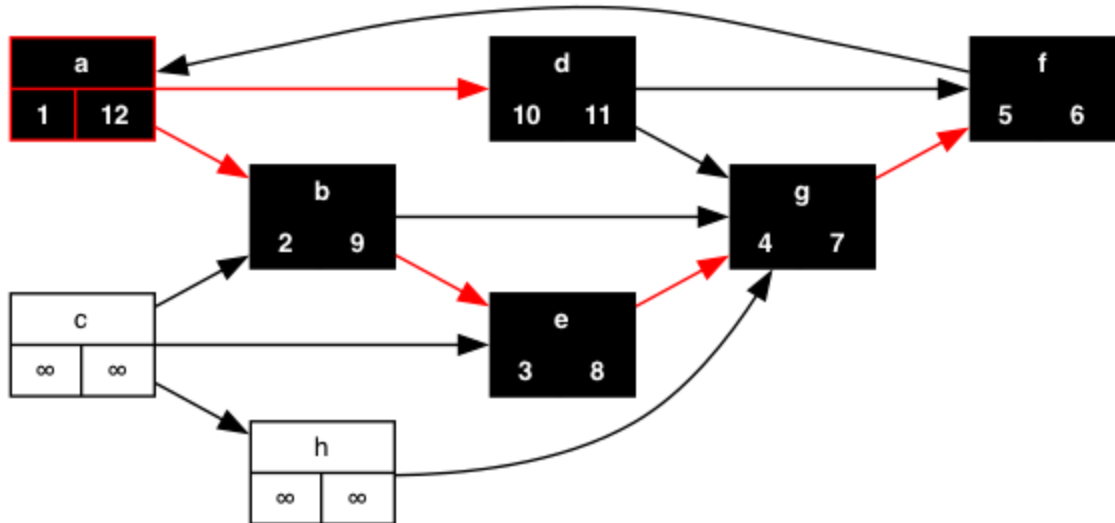
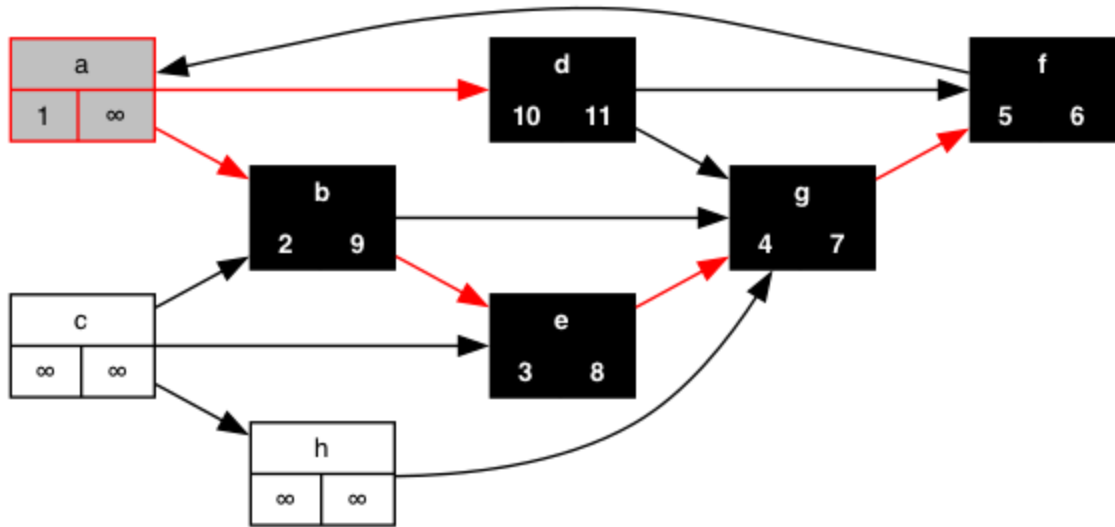
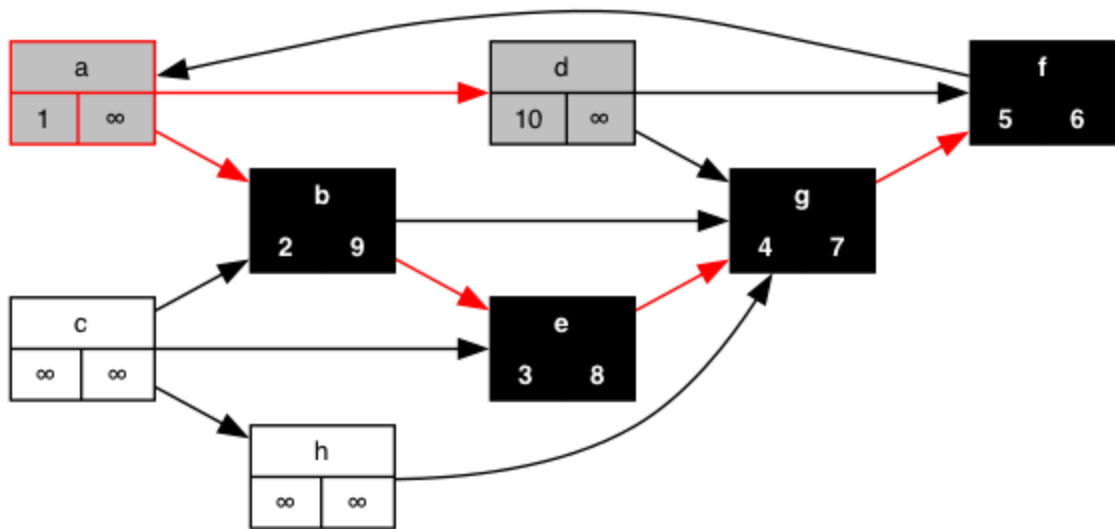
```

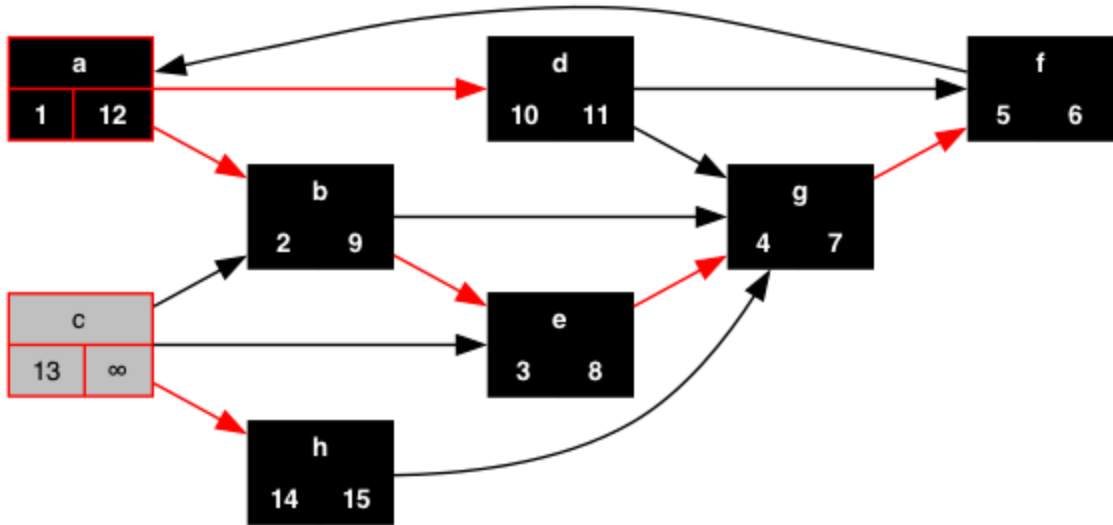
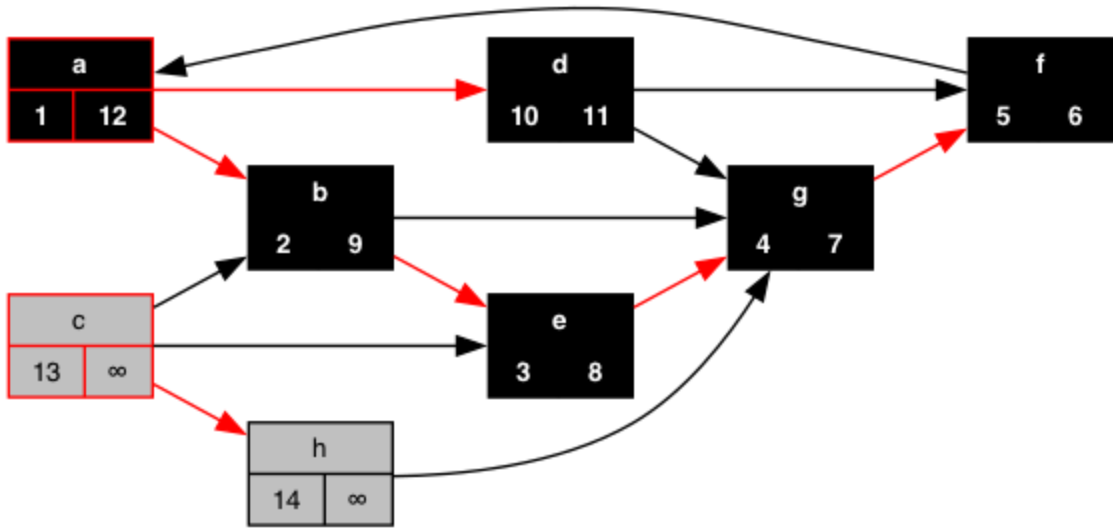
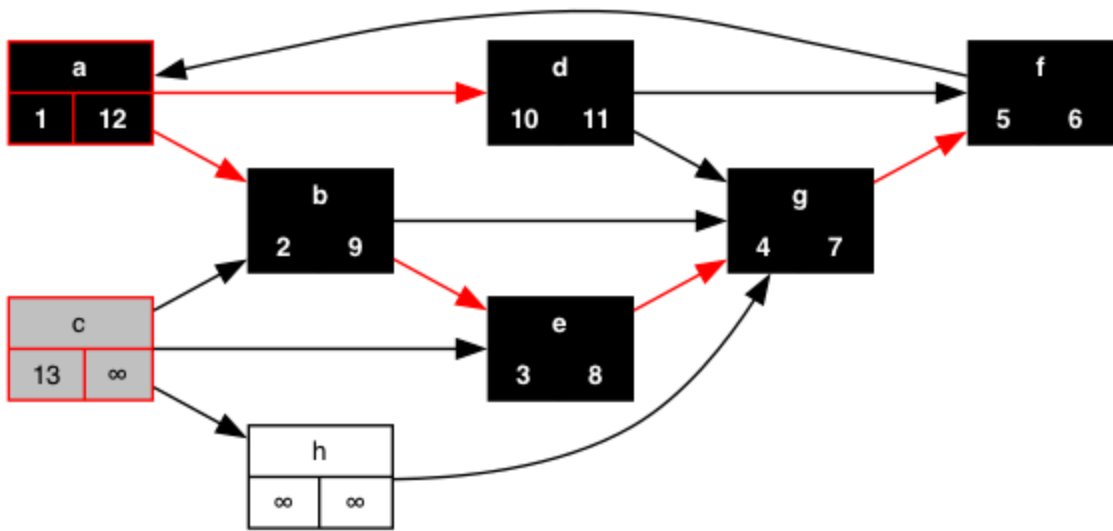


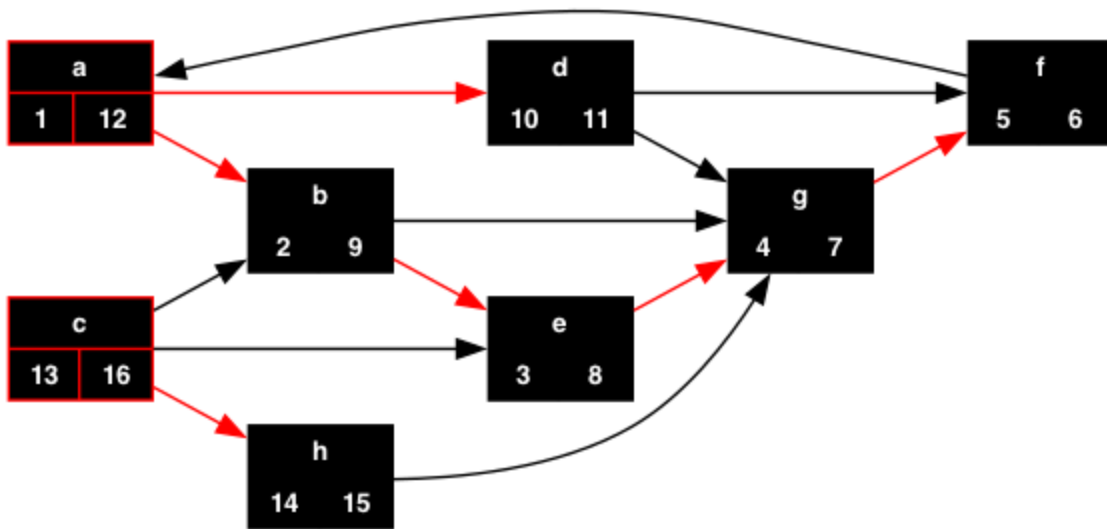






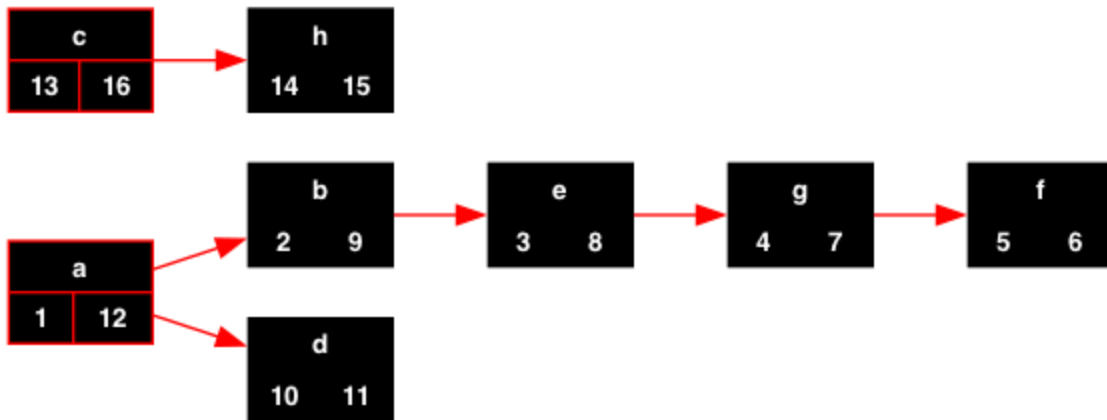






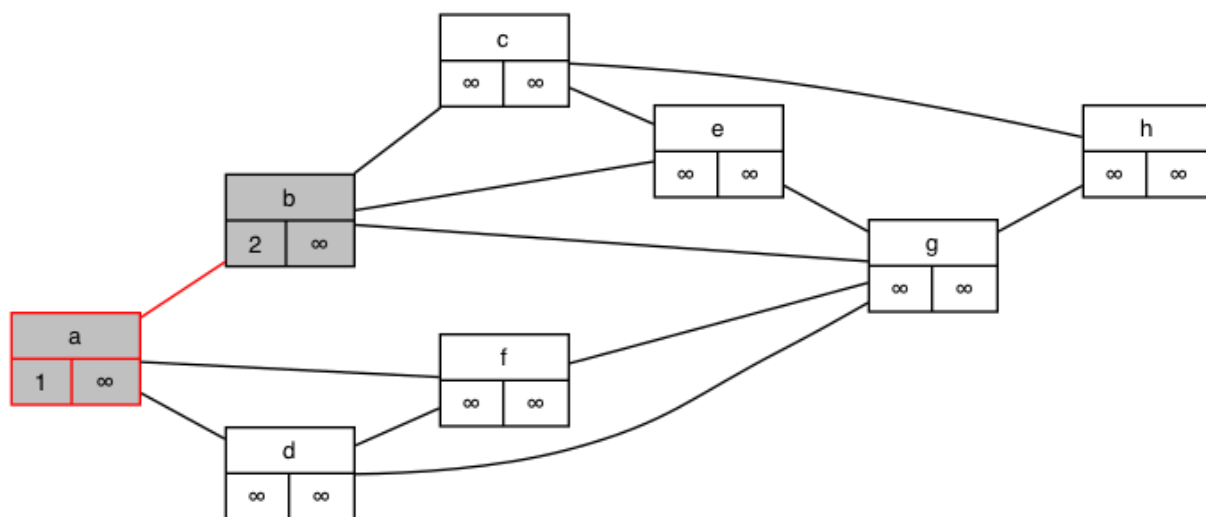
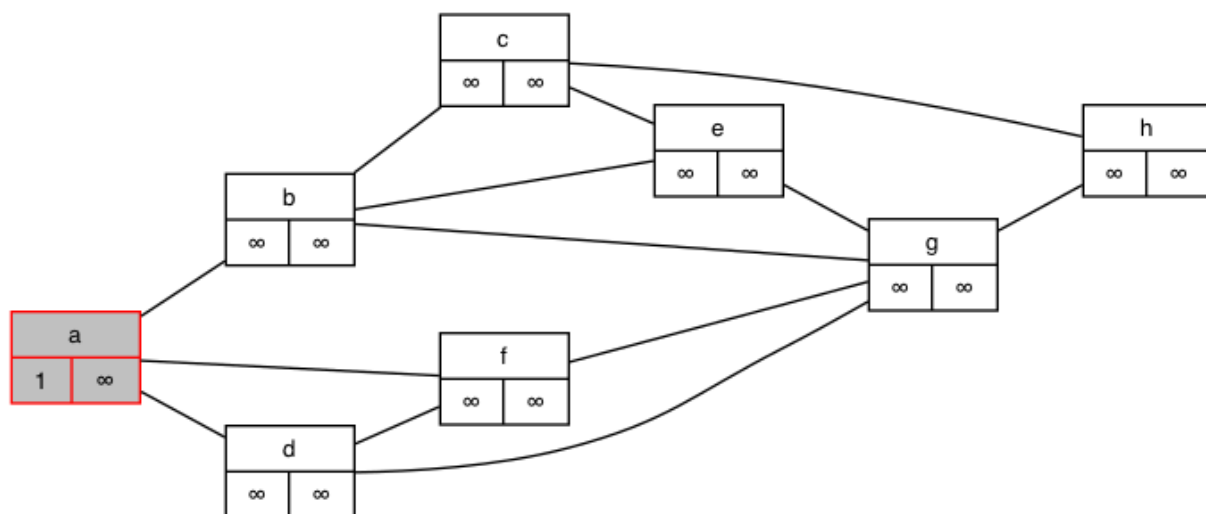
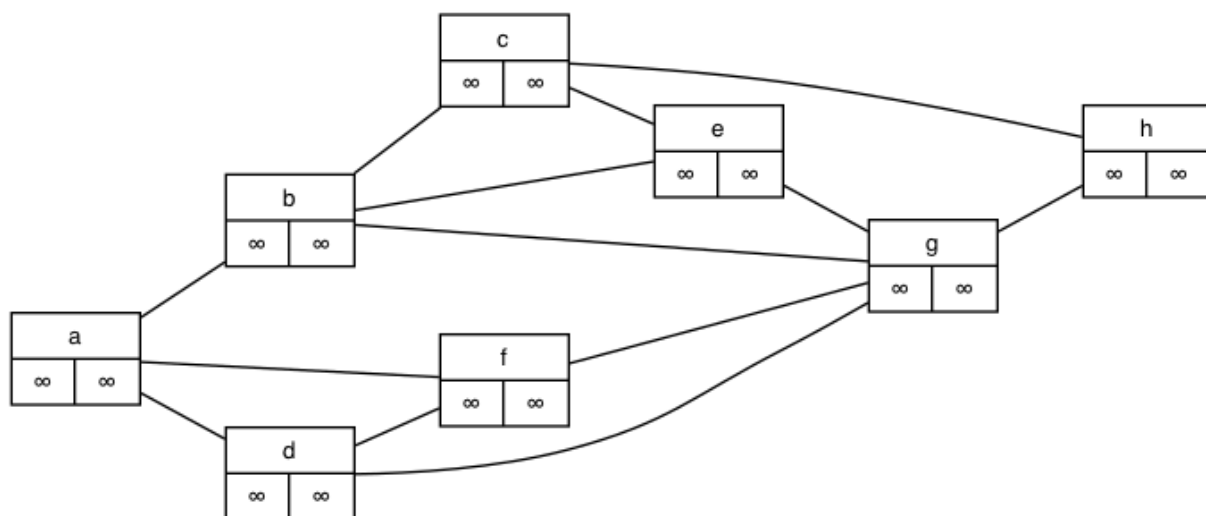
Show resulting forest

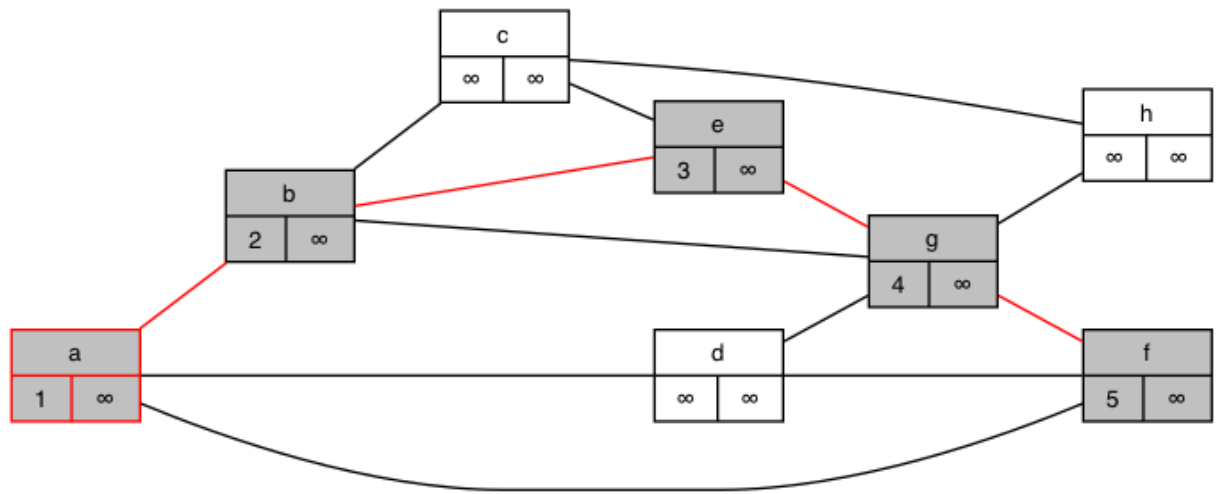
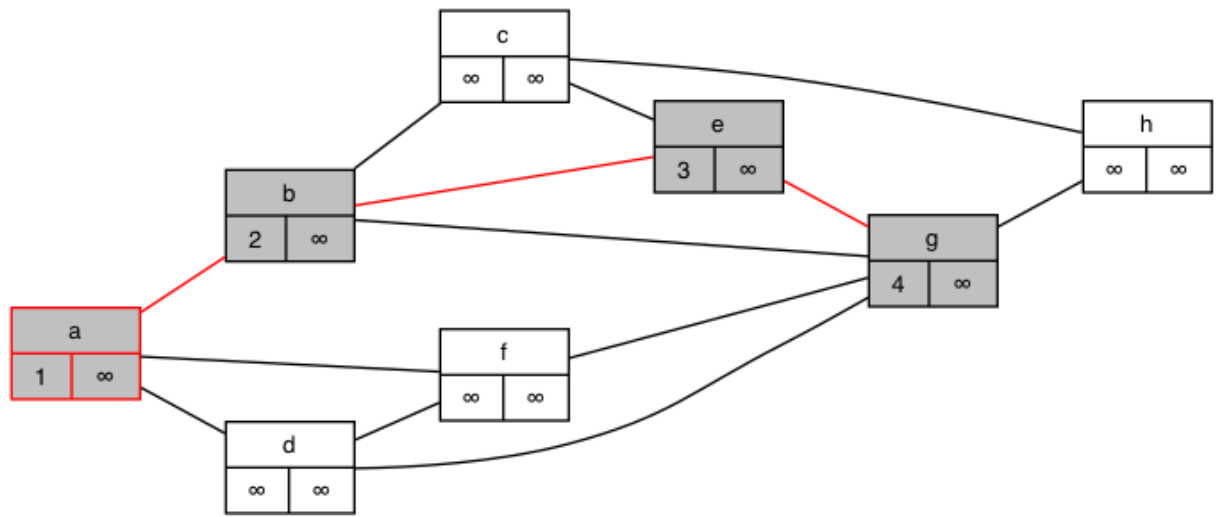
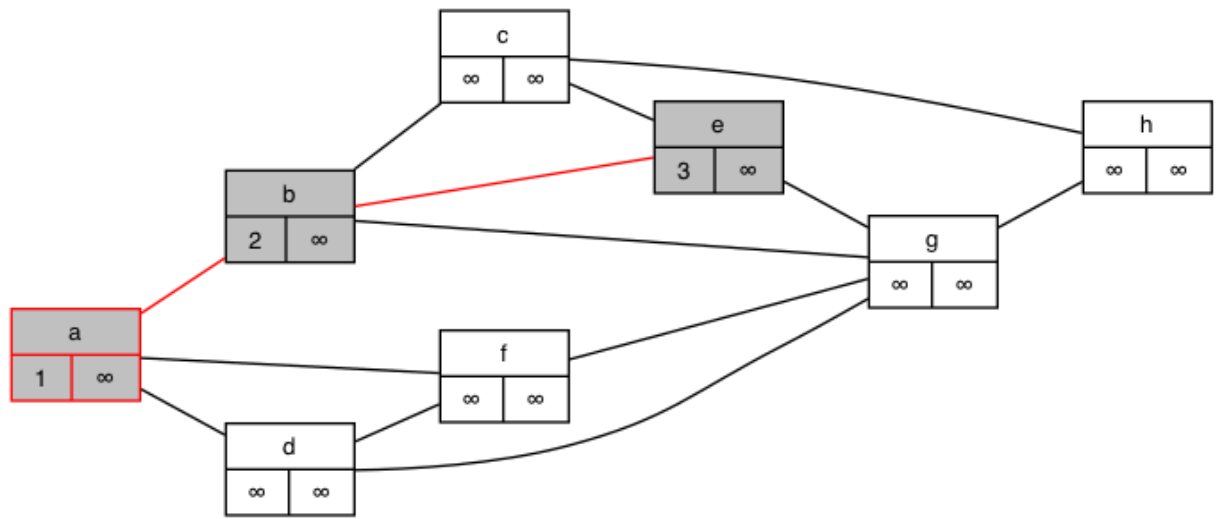
```
In [7]: G.render_forest()
```

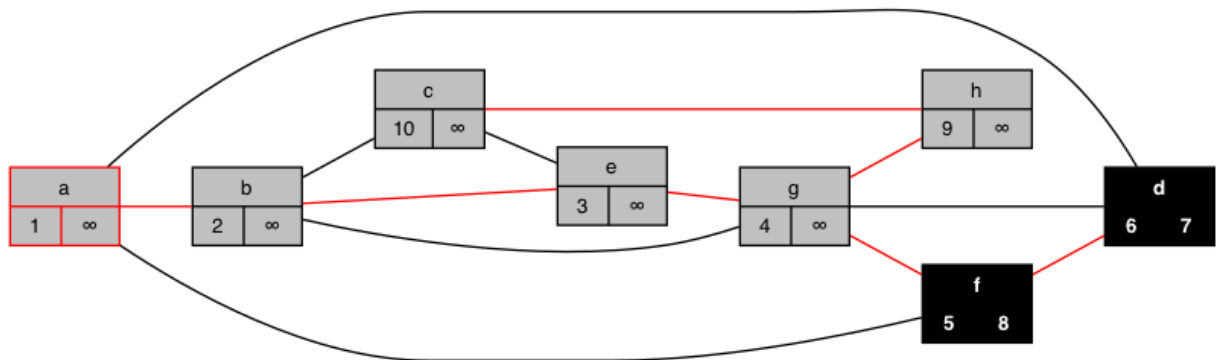
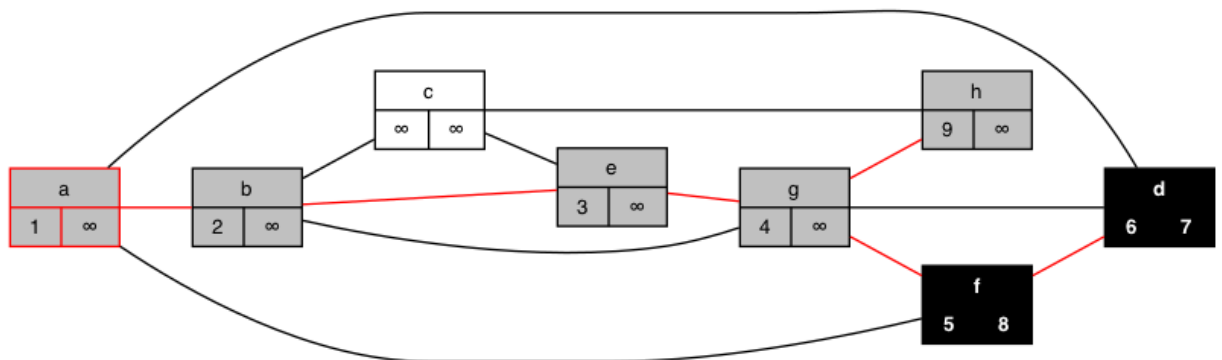
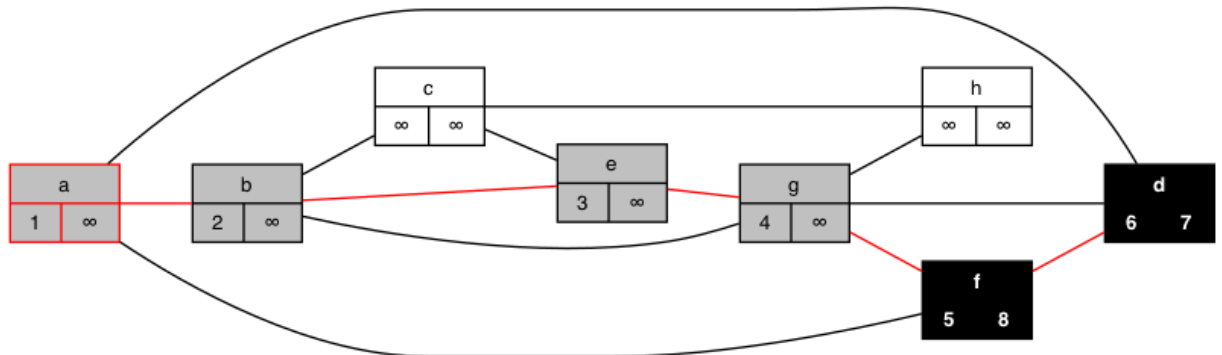
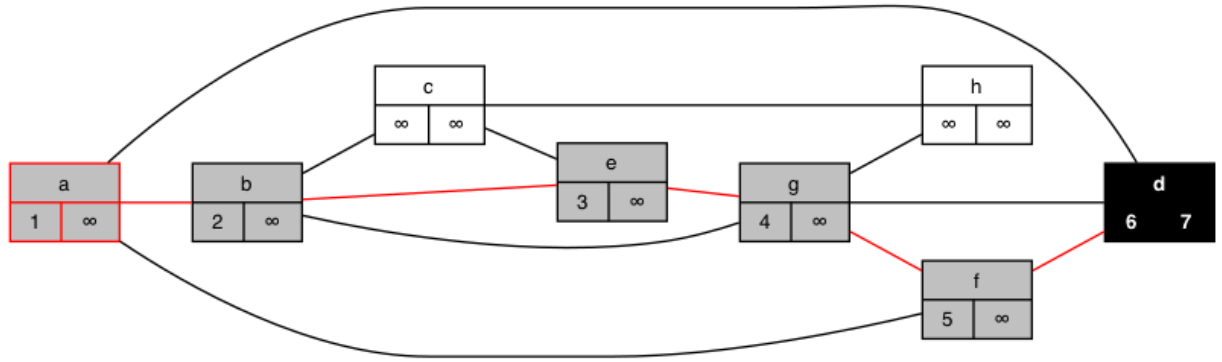
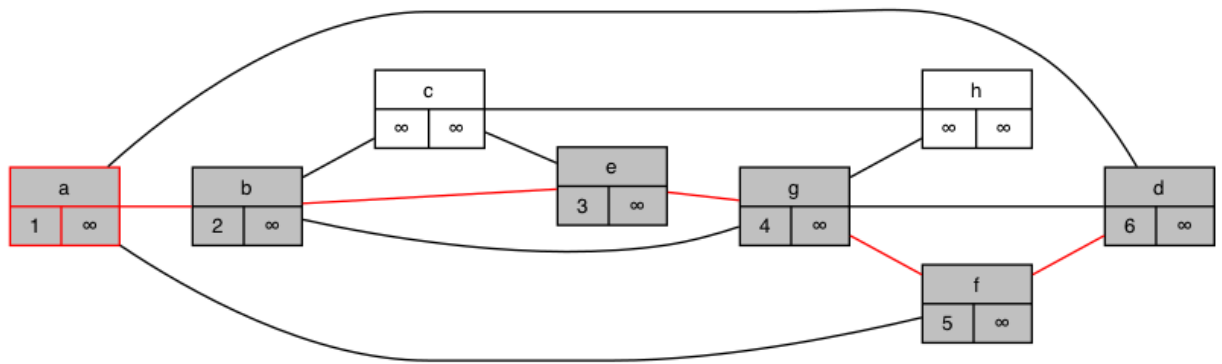


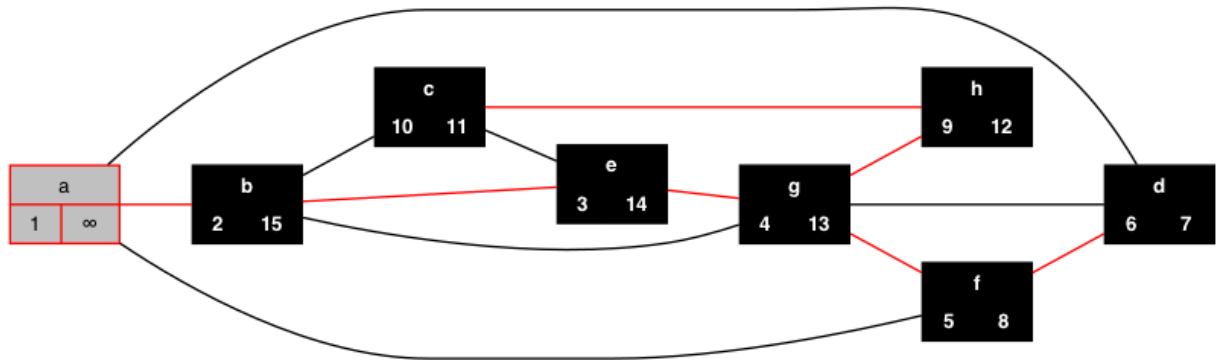
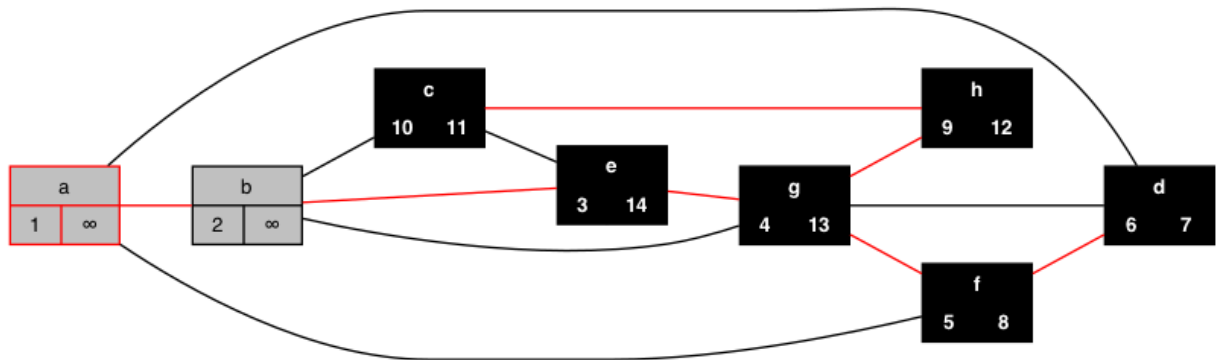
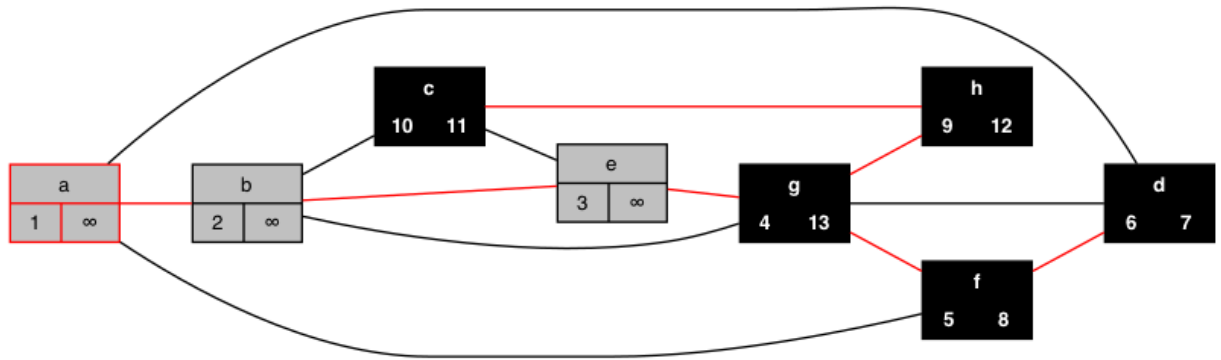
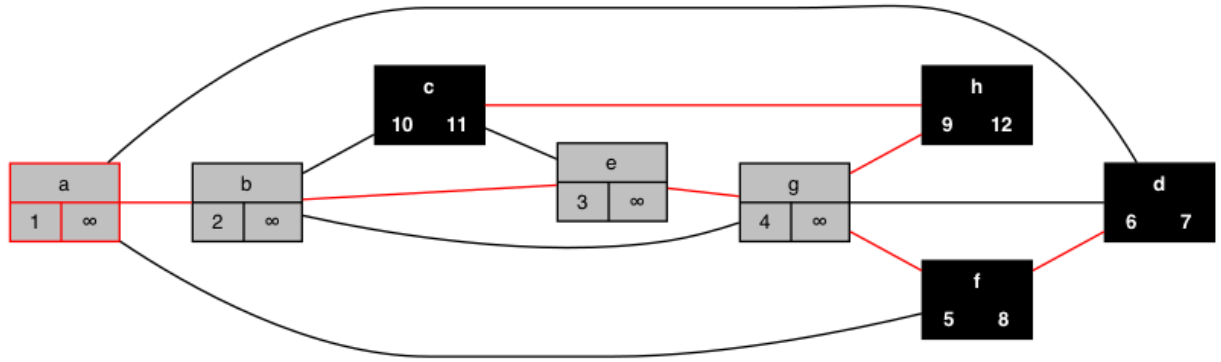
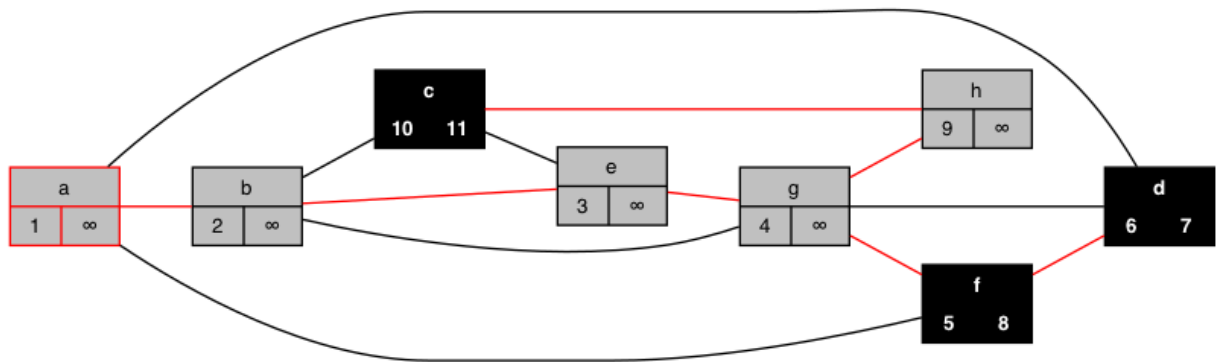
DFS on an undirected graph

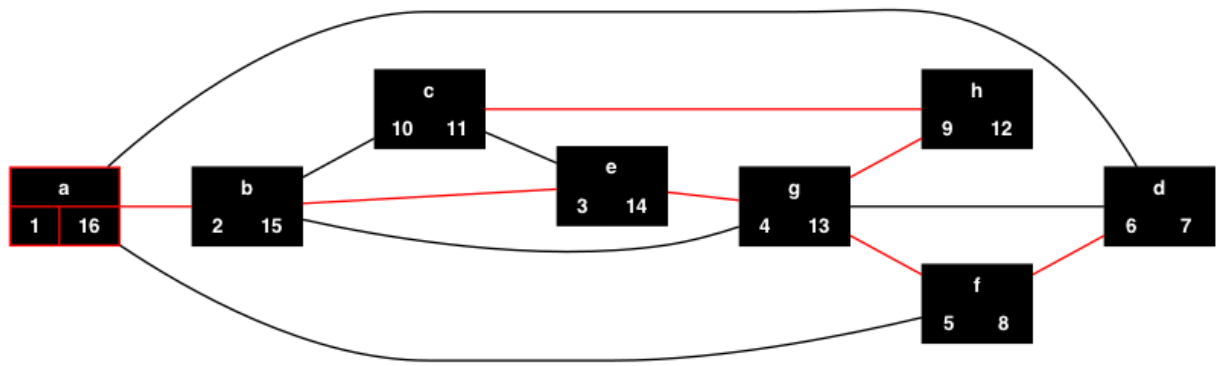
```
In [7]: UG = DFSGraph(Gal, directed=False)
        UG.run_dfs()
```



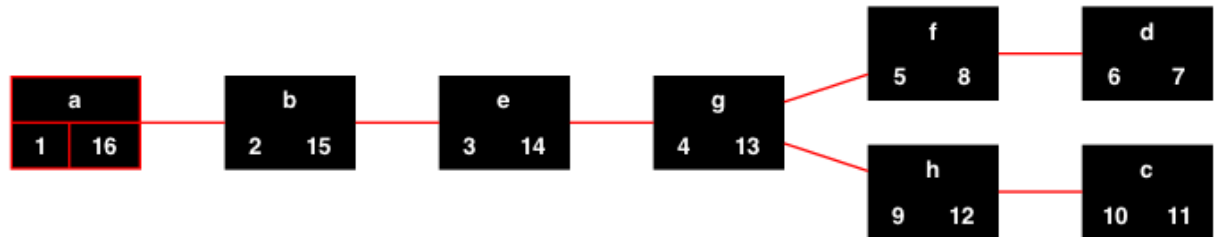








```
In [9]: UG.render_forest()
```



Topological sort

Compendium, Part 19; Cormen, Ch 5 (using a different algorithm)

- Sort directed acyclical graphs (DAGs)
- Sorting of graphs with cycles is not possible
- Suitable for situations with *partial order*
 - $a > b$ and $b > c \implies a > c$
 - but we can have a, b, c so that neither $a > b$ nor $b > c$
- Typical for certain types of dependencies, e.g., in project planning

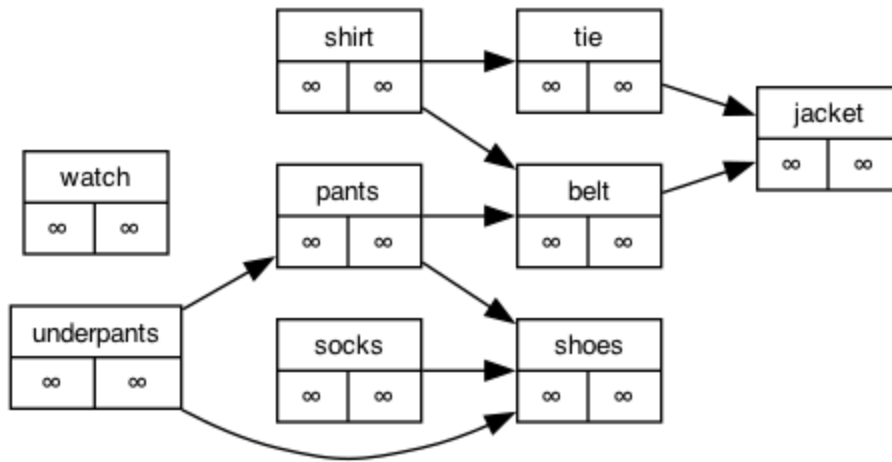
Example: Getting dressed (in order)

The dressing DAG:

- If there is an edge $A \rightarrow B$, then A must be put on before B

```
In [8]: DressAL = {'pants': ['belt', 'shoes'],
                  'belt': ['jacket'],
                  'shirt': ['belt', 'tie'],
                  'tie': ['jacket'],
                  'jacket': [],
                  'socks': ['shoes'],
                  'underpants': ['pants', 'shoes'],
                  'shoes': [],
                  'watch': []}

DG = DFSGraph(DressAL)
DG.render()
```



Idea

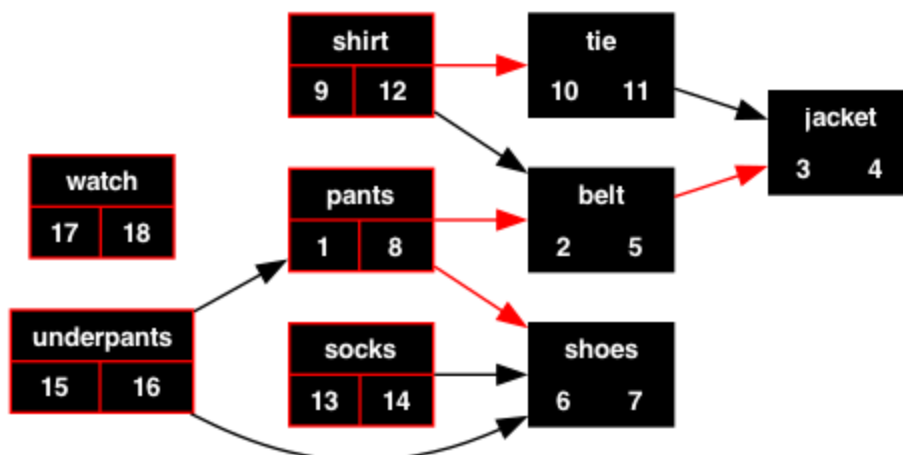
1. Read each arrow in the graph above as "required by".
2. We can execute a task only after its requirements have been fulfilled.
3. Exploring a vertex u means to look for *all* tasks that depend on (require) u .
4. We register the finishing time of u precisely when we have found all tasks requiring u .
5. All tasks depending on u must thus have earlier finishing times, but must be executed *after* u .
6. Thus, executing tasks in order of *descending* finishing times will ensure that all requirements are fulfilled.

Implementation

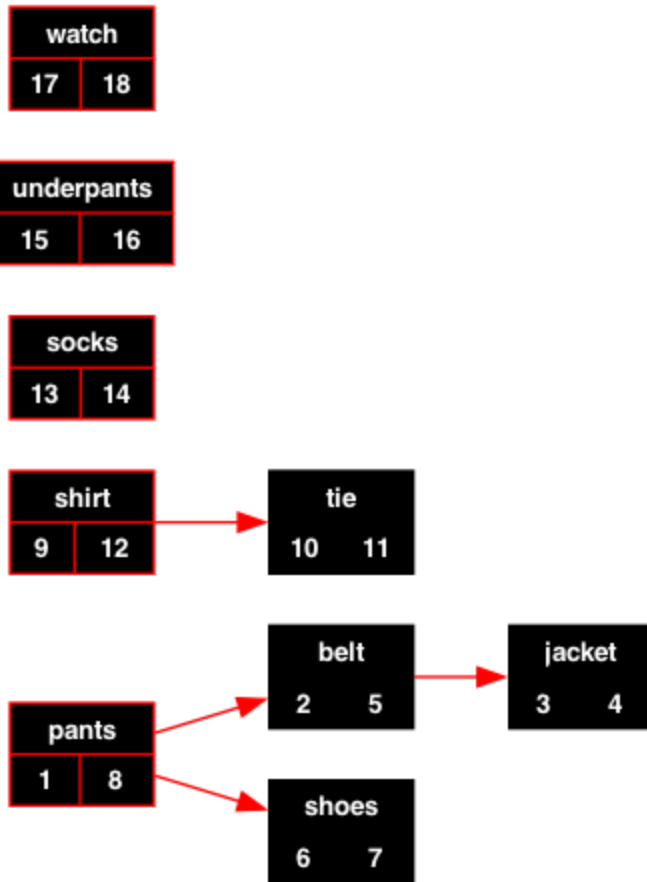
- Depth-first search
- Whenever a vertex is finished, add it to the *beginning* of the sorted list of vertices
- This will result in list of vertices sorted in descending order of finishing times

Plain depth-first search

```
In [9]: DG.run_dfs(silent=True)
        DG.render()
```



```
In [10]: DG.render_forest()
```



Depth-first search augmented with topologically ordered vertex list

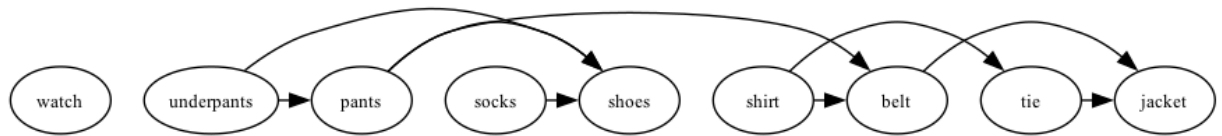
```
In [12]: class TopoSortGraph(DFSGraph):
    def __init__(self, adj_lists):
        super().__init__(adj_lists)
        self.V_sorted = []

    def dfs_visit(self, u):
        super().dfs_visit(u)
        self.V_sorted.insert(0, u)  # insert u in pos 0, i.e., at head of list

    def render_sorted(self):
        # With a little help from perplexity.ai for left-right ordering in subgraph
        g = pydot.Dot(graph_type='digraph', rankdir='TB')
        s = pydot.Subgraph(rank='same', rankdir='TB')
        for u in self.V_sorted:
            s.add_node(u.render(plain=True))
        for adj in self.adj.values():
            for e in adj:
                s.add_edge(e.render(plain=True))
        g.add_subgraph(s)
        IPd.display(IPd.Image(g.create_png()))
```

```
In [13]: DG = TopoSortGraph(DressAL)
         DG.run_dfs(silent=True)
```

```
DG.render_sorted()
```



Not quite the way I would get dressed, but all constraints fulfilled!

Limits of computability

See also Skiena Ch 11.

See PDF file [INF221_V25_L13_PNP.pdf](#) .